

INTRODUCTION TO ALGORITHMS

Algorithms form the foundation of Computer Science and software engineering. Every computational task—from searching the internet and processing transactions to artificial intelligence and scientific simulations—relies on algorithms. In the study of *Algorithm Analysis and Design*, understanding algorithms and their properties is essential before moving to advanced design techniques and complexity analysis.

WHAT IS AN ALGORITHM?

An **algorithm** is a finite sequence of well-defined computational steps that transforms input into output to solve a particular problem.

In simpler terms:

An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.

An algorithm accepts some **input**, processes it through a sequence of instructions, and produces the required **output**.

FORMAL DEFINITION

An algorithm can be viewed as:

Algorithm = Input + Processing Steps + Output

The algorithm must specify:

- What inputs are expected
- How those inputs are processed
- What outputs are generated
- The exact order of execution

EXAMPLE OF AN ALGORITHM

Problem: Find the Largest of Two Numbers

Algorithm

1. Start
2. Read two numbers a and b
3. If $a > b$, print a
4. Otherwise print b
5. Stop

This example demonstrates the essential structure of an algorithm:

- Input
- Processing
- Decision making
- Output

IMPORTANCE OF ALGORITHMS

Algorithms are central to computing because they determine:

- **Efficiency of programs**
- **Optimal use of resources**
- **Execution speed**
- **Memory utilization**
- **Scalability of applications**

Efficient algorithms are critical in areas such as:

- Database systems
- Operating systems
- Compiler design
- Computer networks
- Machine learning
- Cryptography
- Artificial intelligence
- Scientific computing

RELATIONSHIP BETWEEN ALGORITHM AND PROGRAM

An algorithm is different from a computer program.

Algorithm	Program
Logical solution to a problem	Implementation of the algorithm
Language independent	Written in a programming language
Focuses on steps and logic	Focuses on syntax and execution
Abstract concept	Executable entity

Thus:

An algorithm is the blueprint, while a program is the actual construction.

CHARACTERISTICS OF AN ALGORITHM

A valid algorithm possesses several important characteristics. According to standard algorithm textbooks, the following properties are fundamental.

1. INPUT

An algorithm should accept **zero or more inputs**.

Inputs are externally supplied values that are processed by the algorithm.

Examples	Illustration
- Numbers - Strings - Arrays - Graphs	For a sorting algorithm: - Input: List of numbers - Output: Sorted list

2. OUTPUT

An algorithm must produce **at least one output**.

The output is the result obtained after processing the input.

Examples	Without output, the computational process has no meaningful purpose.
- Sorted array - Search result - Maximum value - Path in a graph	

3. DEFINITENESS (UNAMBIGUITY)

Every step in an algorithm must be **precisely and clearly defined**.

There should be no ambiguity or confusion in interpretation.

Ambiguous Statement “Process the data efficiently.” Definite Statement “Compare adjacent elements and swap if necessary.”	A computer cannot interpret vague instructions; therefore, each operation must be exact.
---	--

4. FINITENESS

An algorithm must terminate after a **finite number of steps**. It should not run indefinitely.

Example

A loop such as: *while*($x > 0$)

must eventually make $x = 0$ or the algorithm becomes non-terminating.

Termination is essential for practical computation.

5. EFFECTIVENESS

Each operation in the algorithm must be **basic and feasible**.

The instructions should be executable in finite time using available computational resources.

Effective Operation

- Addition
- Comparison
- Assignment

Ineffective Operation

- “Guess the correct answer magically”

Every step should be realistic and computable.

6. CORRECTNESS

An algorithm must produce the correct output for every valid input.

Correctness is usually verified using:

- Mathematical proofs
- Testing
- Formal verification

Example

A sorting algorithm is correct only if:

- All elements are present
- Elements are arranged in proper order

7. GENERALITY

A good algorithm should solve a **general class of problems**, not just a single instance.

Example

A sorting algorithm should sort:

- 10 numbers
- 100 numbers
- Any valid list size

rather than working only for a fixed dataset.

8. EFFICIENCY

An algorithm should use:

- Minimum execution time
- Minimum memory

Efficiency is measured using:

- **Time Complexity**
- **Space Complexity**

This becomes extremely important for large-scale applications.

9. DETERMINISM

In a deterministic algorithm:

- The same input always produces the same output
- Execution steps remain predictable

Example

Binary Search is deterministic.

10. LANGUAGE INDEPENDENCE

Algorithms are independent of programming languages.

The same algorithm can be implemented in:

- C
- C++
- Java
- Python

Thus, algorithms focus on logic rather than syntax.

FUNDAMENTAL STRUCTURE OF AN ALGORITHM

Most algorithms follow this general structure:

Algorithm Problem_Name

Input
Processing
Output

REPRESENTATION OF ALGORITHMS

Algorithms can be represented in several forms.

1. NATURAL LANGUAGE

Uses ordinary English statements.

Advantages

- Easy to understand

Disadvantages

- Can become ambiguous

2. PSEUDOCODE

A structured, programming-like notation without strict syntax rules.

Example

```
Algorithm LinearSearch(A, n, key)

for i = 1 to n
    if A[i] = key
        return i

return -1
```

Advantages

- Easy to read
- Language independent
- Widely used in textbooks

3. FLOWCHARTS

Graphical representation of algorithms using symbols.

Common symbols include:

Symbol	Meaning
Oval	Start/Stop
Parallelogram	Input/Output
Rectangle	Process
Diamond	Decision

QUALITIES OF A GOOD ALGORITHM

A good algorithm should be:

- Correct
- Simple

- Readable
- Efficient
- Modular
- Scalable
- Robust

ALGORITHM DESIGN GOALS

The primary goals in algorithm design are:

1. Correctness
2. Efficiency
3. Simplicity
4. Scalability
5. Maintainability

TYPES OF ALGORITHMS

Algorithms can be classified based on design methodology.

Common Categories

Type	Example
Brute Force	Linear Search
Divide and Conquer	Merge Sort
Greedy Method	Dijkstra's Algorithm
Dynamic Programming	Floyd-Warshall
Backtracking	N-Queens Problem
Branch and Bound	Traveling Salesman
Randomized Algorithms	Randomized Quick Sort

ALGORITHM ANALYSIS

Algorithm analysis evaluates resource requirements.

Two major measures are:

Time Complexity

Measures execution time as input size increases.

Common notations:

- $O(n)$
- $O(n^2)$
- $O(\log n)$

Space Complexity

Measures memory usage during execution.

Includes:

- Fixed space
- Variable space
- Auxiliary space

CRITERIA FOR ANALYZING ALGORITHMS

Algorithm analysis is the process of evaluating an algorithm with respect to the resources it requires and its performance characteristics. The primary goal is to determine how efficiently an algorithm solves a problem as the size of the input increases.

Different algorithms can solve the same problem, but their efficiencies may vary greatly.

For example:

- One sorting algorithm may take seconds.
- Another may take hours for large datasets.

Algorithm analysis helps us:

- Compare alternative algorithms
- Predict performance
- Select optimal solutions
- Understand scalability
- Estimate resource consumption

MAJOR CRITERIA FOR ANALYZING ALGORITHMS

The important criteria used in algorithm analysis are:

1. Time Complexity
2. Space Complexity

1. TIME COMPLEXITY

Definition

Time complexity measures the amount of computational time required by an algorithm as a function of input size.

If input size is denoted by n , then time complexity expresses how running time grows as n increases.

Importance

Time is usually the most critical resource in computation.

Efficient algorithms:

- Execute faster
- Handle large datasets
- Improve system responsiveness

Measuring Time Complexity

Instead of actual clock time, algorithm analysis counts:

- Number of comparisons
- Number of assignments
- Number of arithmetic operations
- Number of loop iterations

This machine-independent approach gives theoretical efficiency.

Example: Linear Search

```
for i = 1 to n
  if A[i] = key
    return i
```

If the element is not found, the loop executes n times.

Therefore: $T(n) = n$

Time complexity: $O(n)$

Cases in Time Complexity

Best Case

Minimum execution time.

Example

In linear search:

- Key found at first position

$$T(n) = 1$$

Worst Case

Maximum execution time.

Example

Key found at last position or absent.

$$T(n) = n$$

Worst-case analysis is most commonly used.

Average Case

Expected execution time over all possible inputs.

Usually more realistic but mathematically harder.

Asymptotic Notations

Algorithm efficiency is represented using asymptotic notations. (will study in detail later)

Big-O Notation

Upper bound of growth rate.

$$T(n) = O(f(n))$$

Represents worst-case complexity.

Omega Notation

Lower bound.

$$T(n) = \Omega(f(n))$$

Represents best-case complexity.

Theta Notation

Tight bound.

$$T(n) = \Theta(f(n))$$

Represents exact asymptotic growth.

Common Time Complexities

Complexity	Meaning	Example
$O(1)$	Constant	Array access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Traveling Salesman (Brute Force)

2. SPACE COMPLEXITY

Definition

Space complexity measures the total memory required by an algorithm during execution.

It includes:

- Input space
- Auxiliary space
- Recursion stack space
- Dynamic memory allocation

Components of Space Complexity

Fixed Part

Independent of input size.

Includes:

- Program instructions
- Constants
- Simple variables

Variable Part

Depends on input size.

Includes:

- Arrays
- Dynamic structures
- Recursive stack frames

Example

```
int sum = 0;
for i = 1 to n
    sum = sum + A[i];
```

Extra memory used:

- sum
- i

Only constant extra space is used.

Therefore:

$$S(n) = O(1)$$

Importance of Space Complexity

Efficient memory usage is essential in:

- Embedded systems
- Mobile devices
- Real-time systems
- Large-scale computations

Sometimes a trade-off exists between time and space.

MACHINE-INDEPENDENT ANALYSIS

One important principle in algorithm analysis is that efficiency should be independent of:

- Programming language
- Computer hardware
- Operating system

Thus, asymptotic analysis is preferred over actual timing measurements.

Empirical vs Theoretical Analysis

Theoretical Analysis	Empirical Analysis
Mathematical	Experimental
Machine independent	Machine dependent
Uses asymptotic notation	Uses actual execution time
Predicts scalability	Measures real performance

Both approaches are important.

TIME-SPACE TRADE-OFF

Sometimes algorithms reduce time by using more memory.

Example

Hash tables:

- Faster searching
- Additional memory required

This is called a **time-space trade-off**.

CRITERIA FOR A GOOD ALGORITHM

A good algorithm should ideally satisfy:

Criterion	Requirement
Correctness	Produces accurate output
Efficiency	Uses minimal resources
Finiteness	Terminates properly
Simplicity	Easy to understand
Scalability	Handles large inputs
Robustness	Handles edge cases
Optimality	Best possible performance

PERFORMANCE ANALYSIS

Performance analysis means studying how efficient an algorithm or data structure is.

Mainly, we check two things:

- **Time Complexity** — How much **time** (number of operations) the algorithm takes.
- **Space Complexity** — How much **memory** (space) the algorithm uses.

Goal:

To **predict** how the algorithm will behave as the input size grows **without running** the code.

1. TIME COMPLEXITY

- Measures the **amount of time** an algorithm takes to complete.
- It is expressed as a function of the **input size n**.
- We usually use **Big-O notation** (e.g., $O(n)$, $O(\log n)$, $O(n^2)$) to express it.

Why it matters?

Because we want algorithms that are **fast even for large inputs**.

Example for Time Complexity:

Suppose we have a simple loop:

```
for (int i = 0; i < n; i++) {
    printf("%d ", i);
}
```

- The loop runs **n times**.
- Each printf is a constant-time operation.
- So, **Time Complexity = $O(n)$** (linear time).

Types of Common Time Complexities:

Complexity	Name	Example
$O(1)$	Constant time	Accessing array element
$O(\log n)$	Logarithmic time	Binary search
$O(n)$	Linear time	Traversing an array
$O(n \log n)$	Linearithmic time	Merge sort, Heap sort
$O(n^2)$	Quadratic time	Bubble sort, Selection sort
$O(2^n)$	Exponential time	Solving Tower of Hanoi

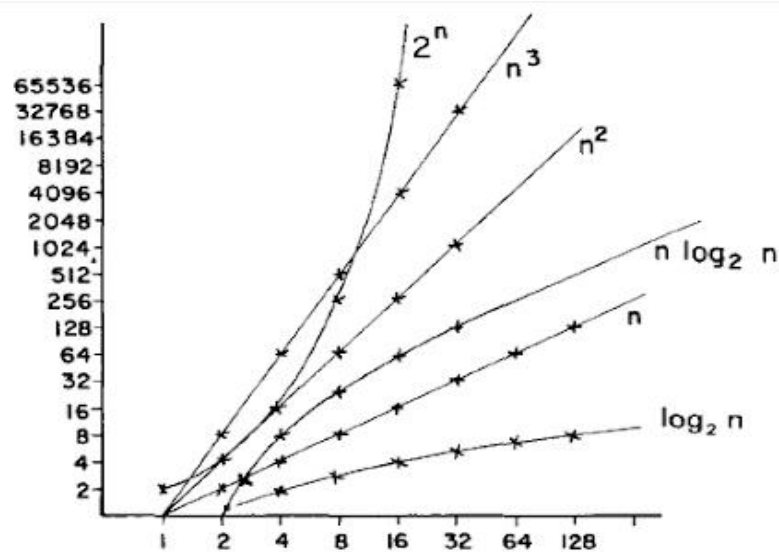


Figure 1.6: Rate of Growth of Common Computing Time Functions

$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	2, 147, 483, 648

Figure 1.7: Values for Computing Functions

2. SPACE COMPLEXITY

- Measures the **amount of memory** an algorithm needs to complete its execution.
- Memory includes:
 - Variables
 - Data structures (arrays, linked lists)
 - Recursion stack space

Why it matters?

Because **limited memory** can slow down or crash a program.

Example for Space Complexity:

Suppose:

```
int a, b, c; // 3 integers
```

- These use **constant memory** → Space Complexity = **O(1)**.

Now:

```
int arr[n]; // array of n elements
```

- Memory used grows with **n** → Space Complexity = **O(n)**.

Example Combining Both:

Suppose we want to find the sum of all elements in an array:

```
int sum(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += arr[i];
    }
    return total;
}
```

• Time Complexity:

The loop runs **n** times → **O(n)**.

• Space Complexity:

Only total variable used (ignoring input array) → **O(1)**.

Summary Table

Aspect	Description	Example
Time Complexity	How long an algorithm runs	$O(n)$, $O(\log n)$, $O(n^2)$
Space Complexity	How much memory is needed	$O(1)$, $O(n)$
Performance Analysis	Study of time and space needs	Done before execution

Why Performance Analysis is Important?

- Helps to **choose the best algorithm**.
- Helps to **predict performance** for large inputs.
- Avoids wasting time, memory, and power.

Simple C Program To Demonstrate Time Complexity

We'll create a program to **traverse an array** — this has **$O(n)$** time complexity. We'll also measure the **execution time** using `<time.h>` functions.

```
#include <stdio.h>
#include <time.h>

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    // Filling the array
    for (int i = 0; i < n; i++) {
        arr[i] = i;
    }
    clock_t start, end;
    double cpu_time_used;
    // Start the clock
    start = clock();
    // Traversing the array ( $O(n)$  operation) and finding the sum of elements
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    // End the clock
    end = clock();
    // Calculate time taken
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Sum of elements = %d\n", sum);
    printf("Time taken = %f seconds\n", cpu_time_used);

    return 0;
}
```

Explanation

- clock_t start, end; — to record start and end time.
- clock() — function returns processor time consumed so far.
- The for loop is **O(n)** — it depends linearly on the size n.
- We find the **difference** between start and end to measure how long it took.

You can enter a **large value of n** (like 1000000) and see how time increases!

C Program for Bubble Sort with Time Measurement

```
#include <stdio.h>
#include <time.h>

void bubbleSort(int arr[], int n) {
    int temp;
    for (int i = 0; i < n - 1; i++) {          // n-1 passes
        for (int j = 0; j < n - i - 1; j++) {  // comparisons in each pass
            if (arr[j] > arr[j + 1]) {        // swap if elements are out of order
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    // Filling array in reverse order to make it worst case
    for (int i = 0; i < n; i++) {
        arr[i] = n - i;
    }
}
```

```

clock_t start, end;
double cpu_time_used;
start = clock();
bubbleSort(arr, n);
end = clock();
cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("Array sorted using Bubble Sort.\n");
printf("Time taken = %f seconds\n", cpu_time_used);
return 0;
}

```

Output

Enter number of elements: 100

Array sorted using Bubble Sort.

Time taken = 0.000029 seconds

Explanation

- **Outer loop:** Runs $n-1$ times.
- **Inner loop:** On average $n/2$ times for each pass.
- Thus, **total comparisons** $\approx n^2/2 \Rightarrow O(n^2)$ time complexity.
- If $n = 10000$, it will take a **longer time** compared to simple array traversal ($O(n)$)!

Key Observations

Complexity	Program	Time taken
$O(n)$	Array Traversal	Very fast (even for large n)
(Sum) $O(n^2)$	Bubble Sort	Very slow for large n (say $n > 5000$)

- Bubble Sort is **inefficient** when n is large because of its quadratic nature.
- That's why **better sorting algorithms** like Merge Sort ($O(n \log n)$) are preferred.

Summary

- $O(n)$ — grows linearly, fast for large inputs.
- $O(n^2)$ — grows quadratically, very slow when n increases.
- **Measuring time** helps in **understanding real performance**, not just theoretical Big O

ASYMPTOTIC ANALYSIS AND ASYMPTOTIC NOTATIONS

Asymptotic analysis is one of the most important concepts in Algorithm Analysis and algorithm design. It provides a mathematical framework for evaluating the efficiency of algorithms as the input size becomes very large.

1. INTRODUCTION TO ASYMPTOTIC ANALYSIS**What is Asymptotic Analysis?**

Asymptotic analysis studies the behavior of algorithms as the input size n approaches infinity.

Instead of measuring:

- Actual execution time
- CPU speed
- Hardware performance

asymptotic analysis focuses on:

- Growth rate of running time
- Growth rate of memory usage

Actual running time depends on:

- Processor speed
- Compiler optimization
- Programming language
- System architecture

Therefore, direct timing comparisons are unreliable.

Asymptotic analysis provides:

- Machine-independent evaluation
- Mathematical comparison of algorithms

- Scalability prediction

Fundamental Idea

Suppose two algorithms solve the same problem.

Input Size	Algorithm A	Algorithm B
10	Fast	Fast
1,000	Moderate	Slow
1,000,000	Efficient	Extremely Slow

As input size increases, growth rate becomes more important than constant execution time.

Asymptotic analysis focuses on this growth behavior.

Example

Consider:

$$T(n) = 3n^2 + 5n + 10$$

For large n :

- n^2 dominates
- Lower-order terms become insignificant

Therefore:

$$T(n) = \Theta(n^2)$$

Objectives of Asymptotic Analysis

Asymptotic analysis helps to:

1. Compare algorithms
2. Predict scalability
3. Ignore machine-dependent constants
4. Study worst-case behavior
5. Simplify complexity expressions

2. GROWTH OF FUNCTIONS

In algorithm analysis, running time is expressed as a function of input size n .

Common Growth Rates

Function	Growth Rate
1	Constant
$\log n$	Logarithmic
n	Linear
$n \log n$	Linearithmic
n^2	Quadratic
n^3	Cubic
2^n	Exponential
$n!$	Factorial

Order of Growth

The efficiency ranking from best to worst is:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

Algorithms with smaller growth rates are more efficient for large inputs.

3. ASYMPTOTIC NOTATIONS

Asymptotic notations mathematically describe the limiting behavior of functions.

The three major notations are:

1. Big-O Notation ($O \approx \leq$)
2. Omega Notation ($\Omega \approx \geq$)
3. Theta Notation ($\Theta \approx =$)

Additional notations:

- Little-o ($o \approx <$)
- Little-omega ($\omega \approx >$)

1. Big-O Notation

Definition

Big-O notation gives an **upper bound** on the growth rate of a function.

It represents the worst-case complexity.

Formal Definition

A function $f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n), \forall n \geq n_0$$

Interpretation

Big-O means:

Beyond some point, $f(n)$ does not grow faster than $g(n)$ multiplied by a constant.

Example 1

Consider: $f(n) = 3n + 2$

We show: $f(n) = O(n)$

Because: $3n + 2 \leq 4n$

Thus:

- $c = 4$
- $n_0 = 2$

Example 2

Consider: $f(n) = 5n^2 + 3n + 8$

Dominant term is n^2

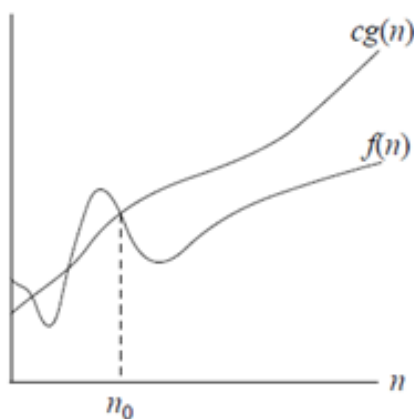
Therefore: $f(n) = O(n^2)$

Example 3

$2n^2 = O(n^3)$, with $c = 1$ and $n_0 = 2$

Graphical Interpretation***O*-notation**

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$.



$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Big-O gives an asymptotic upper boundary.

The function eventually stays below:

$$c \cdot g(n)$$

Characteristics of Big-O

- Represents worst-case behavior
- Ignores constants
- Ignores lower-order terms
- Widely used in practice

Common Big-O Complexities

Complexity	Example
$O(1)$	Array indexing
$O(\log n)$	Binary Search
$O(n)$	Linear Search
$O(n \log n)$	Merge Sort
$O(n^2)$	Bubble Sort
$O(2^n)$	Recursive Fibonacci

2. Omega Notation

Definition

Omega notation gives a **lower bound** on the growth rate.

It represents best-case or minimum growth behavior.

Formal Definition

A function $f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0$$

Interpretation

Omega means:

Beyond some point, $f(n)$ grows at least as fast as $g(n)$.

Example 1

$$f(n) = 3n^2 + 5n + 2$$

Since: $3n^2 + 5n + 2 \geq 3n^2$

Therefore: $f(n) = \Omega(n^2)$

Example 2

Since there exist constants: $c = 1$ and $n_0 = 16$

such that: $\log n \leq \sqrt{n}$

for all: $n \geq 16$ therefore: $\sqrt{n} = \Omega(\log n)$

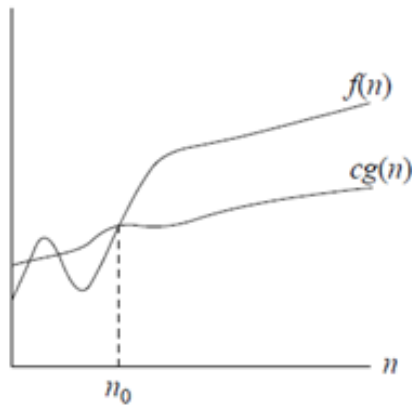
ie;

$$\sqrt{n} = \Omega(\log n), \text{ with } c = 1 \text{ and } n_0 = 16$$

Graphical Interpretation

Ω -notation

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$.



$g(n)$ is an *asymptotic lower bound* for $f(n)$.

3. Theta Notation

Definition

Theta notation gives a **tight bound**.

It represents both:

- Upper bound
- Lower bound

simultaneously.

Formal Definition

A function $f(n)$ is $\Theta(g(n))$ if there exist positive constants c_1, n_0 such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0$$

Interpretation

Theta means:

$f(n)$ grows exactly at the same rate as $g(n)$

Example

$$f(n) = 4n^2 + 3n + 7$$

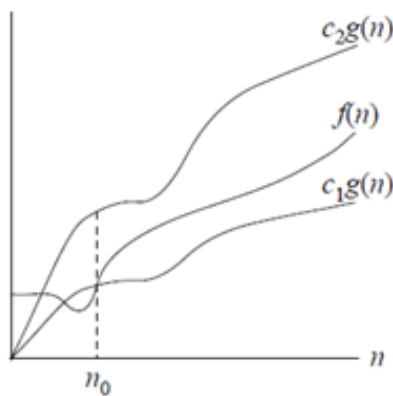
Dominant term: n^2

Hence:

$$f(n) = \Theta(n^2)$$

Graphical Interpretation **Θ -notation**

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$.



$g(n)$ is an *asymptotically tight bound* for $f(n)$.

Relationship Between O, Ω , and Θ

Notation	Meaning
$O(g(n))$	Upper bound
$\Omega(g(n))$	Lower bound
$\Theta(g(n))$	Tight bound

4. Little-o Notation**Definition**

Little-o gives a **strict upper bound**.

if:
$$f(n) = o(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Example $n = o(n^2)$

because:

$$\lim_{n \rightarrow \infty} \frac{n}{n^2} = 0$$

5. Little-Omega Notation

Definition

Little-omega gives a **strict lower bound**.

$$f(n) = \omega(g(n))$$

if:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

Example

$$n^2 = \omega(n)$$

4. PROPERTIES OF ASYMPTOTIC NOTATIONS

Property 1: Reflexive Property

Every function is asymptotically equal to itself.

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

$$f(n) = \Theta(f(n))$$

Property 2: Symmetry Property

If:

$$f(n) = \Theta(g(n))$$

then:

$$g(n) = \Theta(f(n))$$

Property 3: Transitive Property

If:

$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n))$$

then:

$$f(n) = O(h(n))$$

Similarly valid for:

- Ω
- Θ

Property 4: Transpose Symmetry

If:

$$f(n) = O(g(n))$$

then:

$$g(n) = \Omega(f(n))$$

Property 5: Constant Multiplication

If:

$$f(n) = O(g(n))$$

then:

$$cf(n) = O(g(n))$$

for constant $c > 0$

Constants do not affect asymptotic growth.

Property 6: Addition Rule

If:

$$f_1(n) = O(g_1(n)) \text{ and } f_2(n) = O(g_2(n))$$

then:

$$f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$$

Example

$$n^2 + n = O(n^2)$$

because n^2 dominates.

Property 7: Multiplication Rule

If:

$$f(n) = O(g(n)) \text{ and } p(n) = O(q(n))$$

then:

$$f(n)p(n) = O(g(n)q(n))$$

5. BEST, AVERAGE, AND WORST CASE ANALYSIS**Best Case**

Minimum running time.

Example:

- Linear search finds item immediately.

$$T(n) = O(1)$$

Worst Case

Maximum running time.

Example:

- Item at last position.

$$T(n) = O(n)$$

Average Case

Expected running time over all inputs.

Usually more complex mathematically.

6. IMPORTANCE OF ASYMPTOTIC ANALYSIS

Asymptotic analysis helps in:

- Selecting efficient algorithms
- Predicting scalability
- Designing optimized software
- Comparing alternatives
- Understanding computational limits

It is fundamental in:

- Data structures
- Operating systems
- Database systems
- Compiler design

7. ADVANTAGES OF ASYMPTOTIC ANALYSIS

Advantage	Description
Machine Independent	Works across systems
Simplifies Analysis	Ignores constants
Predicts Scalability	Useful for large inputs
Mathematical Precision	Formal comparison possible

8. LIMITATIONS OF ASYMPTOTIC ANALYSIS

Limitation	Description
Ignores constants	Small inputs may behave differently
Ignores hardware effects	Cache and memory not considered
Worst-case emphasis	Practical behavior may vary

COMPARING GROWTH RATES USING LIMITS

One standard method to compare function sizes is to evaluate:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

The value of this limit determines the relative growth of the functions.

Case 1: Limit Equals Zero

If:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

then:

- $f(n)$ grows slower than $g(n)$
- $f(n) = o(g(n))$

Example

Compare:

$$f(n) = n \text{ and } g(n) = n^2$$

$$\text{Compute: } \lim_{n \rightarrow \infty} \frac{n}{n^2} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

$$\text{Hence: } n = o(n^2)$$

So n^2 is asymptotically larger.**Case 2: Limit Equals a Positive Constant**

If:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, 0 < c < \infty$$

then:

- Both functions grow at the same rate
- $f(n) = \Theta(g(n))$

Example

Compare:

$$f(n) = 3n^2 + 2n \text{ and } g(n) = n^2$$

Compute:

$$\lim_{n \rightarrow \infty} \frac{3n^2 + 2n}{n^2} = \lim_{n \rightarrow \infty} \left(3 + \frac{2}{n}\right) = 3$$

Since the limit is a positive constant:

$$f(n) = \Theta(n^2)$$

Case 3: Limit Equals Infinity

If:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

then:

- $f(n)$ grows faster than $g(n)$
- $f(n) = \omega(g(n))$

Example

Compare:

$$f(n) = n^3 \text{ and } g(n) = n^2$$

Compute:

$$\lim_{n \rightarrow \infty} \frac{n^3}{n^2} = \infty$$

$$\text{Therefore: } n^3 = \omega(n^2)$$

Common Growth Rates

The following ordering is fundamental in algorithm analysis:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

Interpretation of Growth Rates

Function	Growth Type	Efficiency
1	Constant	Excellent
$\log n$	Logarithmic	Very Efficient
n	Linear	Efficient
$n \log n$	Linearithmic	Good
n^2	Quadratic	Moderate
n^3	Cubic	Slow
2^n	Exponential	Very Slow
$n!$	Factorial	Impractical

DOMINANT TERM PRINCIPLE

In asymptotic analysis, the highest-order term dominates.

Example 1

$$f(n) = 5n^3 + 4n^2 + 7n + 2$$

Dominant term: n^3

Thus: $f(n) = \Theta(n^3)$

Example 2

$$f(n) = 1000n + 50$$

Dominant term: n

Thus: $f(n) = \Theta(n)$

Ignoring Constants

Constant multipliers do not affect asymptotic growth.

Example

$100n$ and n

have the same asymptotic size:

$$100n = \Theta(n)$$

because constants are ignored.

Comparing Polynomial Functions

For polynomials:

- Highest degree term determines growth.

Example

Compare: n^2 and n^5

Since degree 5 is larger: $n^2 = o(n^5)$

Comparing Logarithmic and Polynomial Functions

Any polynomial grows faster than logarithmic functions.

Example

$$\log n = o(n)$$

because:

$$\lim_{n \rightarrow \infty} \frac{\log n}{n} = 0$$

Comparing Polynomial and Exponential Functions

Exponential functions grow faster than polynomial functions.

Example

$$n^5 = o(2^n)$$

because:

$$\lim_{n \rightarrow \infty} \frac{n^5}{2^n} = 0$$

Comparing Exponential and Factorial Functions

Factorial functions grow faster than exponentials.

Example

$$2^n = o(n!)$$

Summary Table for Comparing Function Sizes

Limit Result	Relationship
0	$f(n) = o(g(n))$
Positive constant	$f(n) = \Theta(g(n))$
∞	$f(n) = \omega(g(n))$

Practical Importance in Algorithms

Comparing function sizes helps in:

- Choosing efficient algorithms
- Predicting scalability
- Understanding performance limits
- Designing optimized systems

Example in Sorting

Algorithm	Complexity
Bubble Sort	$O(n^2)$
Merge Sort	$O(n \log n)$

Since: $n \log n = o(n^2)$

Merge Sort is asymptotically better.

ANALYZING ALGORITHMS -INSERTION SORT

The analysis of algorithms is one of the central topics in Algorithm Analysis. The objective is to predict the computational resources required by an algorithm, primarily:

- Running time
- Memory usage

Among these, **running time analysis** is the most fundamental.

1. WHY ANALYZE ALGORITHMS?

Different algorithms solving the same problem may require vastly different amounts of resources.

For example:

- One sorting algorithm may complete in milliseconds.

- Another may require minutes for the same input size.

Algorithm analysis helps us:

- Compare algorithms objectively
- Predict scalability
- Estimate performance for large inputs
- Select efficient solutions

2. COMPUTATIONAL MODEL: RAM MODEL

To analyze algorithms, we need a mathematical model of computation.

The standard model used in algorithm analysis is the:

Random Access Machine (RAM) Model

Characteristics of RAM Model

1. Sequential Execution

Instructions execute one after another.

There is:

- No parallel execution
- No concurrency

2. Constant-Time Primitive Operations

Each primitive operation takes constant time.

Examples include:

Arithmetic Operations

- Addition
- Subtraction
- Multiplication
- Division
- Modulus

Data Movement

- Load
- Store

- Copy

Control Operations

- Branching
- Function call
- Return

3. Word Size Assumption

When input size is n , integers are assumed to fit within: $c \log n$

bits for some constant $c \geq 1$.

This assumption ensures:

- We can index array elements
- Word size does not grow arbitrarily

3. WHAT IS RUNNING TIME?

The running time of an algorithm is:

The number of primitive operations executed.

Instead of measuring:

- Seconds
- CPU cycles

we count:

- Comparisons
- Assignments
- Arithmetic operations
- Loop iterations

This makes the analysis:

- Machine independent
- Language independent

4. INPUT SIZE

Running time depends on the size of the input.

Examples

Problem	Input Size
Sorting	Number of elements n
Integer multiplication	Number of bits
Graph algorithms	Number of vertices and edges

5. INSERTION SORT ALGORITHM

Insertion sort works similarly to sorting playing cards in hand.

At each step:

- One element is selected
- Inserted into its correct position among previously sorted elements

Pseudocode of Insertion Sort

INSERTION-SORT(A)

for $j = 2$ to $\text{length}(A)$

$\text{key} = A[j]$

$i = j - 1$

 while $i > 0$ and $A[i] > \text{key}$

$A[i + 1] = A[i]$

$i = i - 1$

$A[i + 1] = \text{key}$

6. BASIC IDEA OF ANALYSIS

In algorithm analysis:

- Each statement takes constant time
- Different statements may have different constants

Suppose:

- Line i takes time c_i

Then:

$$\text{Running Time} = \sum (\text{Cost of Statement}) \times (\text{Number of Executions})$$

7. STEP-BY-STEP ANALYSIS OF INSERTION SORT

Assigning Costs and Total Running Time

INSERTION-SORT(A, n)	cost	times
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 <i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1).$$

Important Observation

The value of: t_j

depends on the input.

Thus:

- Running time varies with input arrangement.

8. BEST-CASE ANALYSIS

Best Case Condition

The array is already sorted.

Example: [1, 2, 3, 4, 5]

For every iteration:

- The while condition fails immediately.

Therefore: $t_j = 1$

for all j .

Running Time in Best Case

Substituting: $t_j = 1$

into the running-time formula:

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1)$$

Simplifying:

$$T(n) = an + b$$

for constants a and b .

Best-Case Complexity

Therefore:

$$T(n) = \Theta(n)$$

Insertion sort runs in **linear time** in the best case.

Interpretation

When data is already sorted:

- Very few comparisons occur
- No shifting is needed

Thus insertion sort becomes highly efficient.

9. WORST-CASE ANALYSIS

Worst Case Condition

The array is reverse sorted.

Example: [5, 4, 3, 2, 1]

Each new element must move all the way to the beginning.

Value of t_j

For each iteration: $t_j = j$

because:

- The while loop compares with all previous elements
- One extra comparison occurs when the loop exits

Summations Required

We use the arithmetic series: $\sum_{j=1}^n j$

whose value is: $\sum_{j=1}^n j = \frac{n(n+1)}{2}$

Also: $\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$

Substituting into Running-Time Formula

After substitution and simplification:

$$T(n) = an^2 + bn + c$$

where a, b, c are constants.

Worst-Case Complexity

Thus: $T(n) = \Theta(n^2)$

Insertion sort has quadratic worst-case running time.

Interpretation

In reverse order:

- Every element shifts repeatedly
- Large number of comparisons occur

Hence execution time grows quadratically.

10. AVERAGE-CASE ANALYSIS

Average Input

Assume:

- Input elements are randomly ordered.

On average:

- Each element compares with half of the sorted portion.

Thus:

$$t_j \approx \frac{j}{2}$$

Result

The average-case running time still becomes quadratic:

$$T(n) = \Theta(n^2)$$

although with a smaller constant factor than the worst case.

11. BEST VS AVERAGE VS WORST CASE

Case	Condition	Complexity
Best	Already sorted	$\Theta(n)$
Average	Random order	$\Theta(n^2)$
Worst	Reverse sorted	$\Theta(n^2)$

12. WHY WORST-CASE ANALYSIS?

Algorithm analysis usually emphasizes worst-case running time.

Reasons

1. Provides Guaranteed Upper Bound

Worst case guarantees:

- Algorithm never exceeds this time.

2. Worst Case Often Occurs

Example:

- Searching for absent elements
- Reverse-ordered inputs

3. Average Case Often Similar

Even average-case insertion sort remains: $\Theta(n^2)$

13. ORDER OF GROWTH

To simplify analysis:

- Ignore constants
- Ignore lower-order terms

Focus only on dominant growth.

Example

Suppose:

$$T(n) = 3n^2 + 5n + 10$$

For large n :

- n^2 dominates

Thus:

$$T(n) = \Theta(n^2)$$

WHY IGNORE CONSTANTS?

As n becomes very large:

- Constants become insignificant.

Example:

$100n^2$ and n^2

have the same asymptotic growth.

1. ANALYSIS OF RECURSIVE ALGORITHMS

1. INTRODUCTION

Many algorithms solve a problem by dividing it into smaller subproblems and solving them recursively. The running time of such algorithms is expressed using **recurrence equations**.

Examples

- Merge Sort
- Quick Sort
- Binary Search
- Tower of Hanoi

2. RECURRENCE EQUATIONS

A **recurrence relation** expresses the running time of an algorithm in terms of the running time of smaller instances.

General Form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where:

- a = Number of subproblems
- n/b = Size of each subproblem
- $f(n)$ = Cost of dividing and combining

Example 1: Binary Search

Binary Search reduces the problem size by half.

$$T(n) = T(n/2) + 1$$

Example 2: Merge Sort

Merge Sort divides into two equal parts and merges them.

$$T(n) = 2T(n/2) + n$$

3. METHODS FOR SOLVING RECURRENCE RELATIONS

3.1 Iteration Method

The recurrence is repeatedly expanded until a recognizable pattern appears.

Example

Solve

$$T(n) = T(n - 1) + 1$$

Expansion

$$\begin{aligned} T(n) &= T(n - 1) + 1 \\ &= T(n - 2) + 1 + 1 \\ &= T(n - 3) + 1 + 1 + 1 \end{aligned}$$

Continuing,

$$= T(1) + (n - 1)$$

If $T(1) = 1$

Then $T(n) = n$

Complexity $T(n) = O(n)$

Example: Binary Search

$$T(n) = T(n/2) + 1$$

Expanding:

$$\begin{aligned} &= T(n/4) + 2 \\ &= T(n/8) + 3 \end{aligned}$$

After k expansions:

$$T(n) = T(n/2^k) + k$$

Stopping when

$$\begin{aligned} n/2^k &= 1 \\ 2^k &= n \\ k &= \log n \end{aligned}$$

Therefore, $T(n) = O(\log n)$

3.2 Recursion Tree Method

A recurrence is represented as a tree.

Each node shows the work done at that recursive call.

Total cost = Sum of costs at all levels.

Example: Merge Sort

$$T(n) = 2T(n/2) + n$$

Level 0

Cost = n

Level 1

Two subproblems:

$$2 \times \frac{n}{2} = n$$

Cost = n

Level 2

$$4 \times \frac{n}{4} = n$$

Cost = n

Each level contributes n .

Number of levels: $\log n$

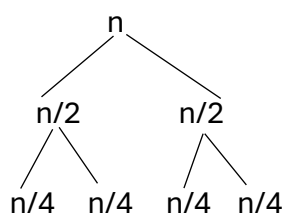
Total cost: $n + n + n + \dots + n$

($\log n$ times)

$$T(n) = n \log n$$

Complexity $O(n \log n)$

Recursion Tree of Merge Sort



Cost per level = n

Depth = $\log n$

Total = $n \log n$

3.3 Substitution Method

The solution is guessed first and then proved using induction.

Example

Given

$$T(n) = 2T(n/2) + n$$

Guess:

$$T(n) = O(n \log n)$$

Assume:

$$T(n/2) \leq c(n/2) \log (n/2)$$

Substitute:

$$\begin{aligned} T(n) &\leq 2[c(n/2) \log (n/2)] + n \\ &= cn \log (n/2) + n \\ &= cn(\log n - 1) + n \\ &= cn \log n - cn + n \end{aligned}$$

Choose suitable constant c .

Hence,

$$T(n) = O(n \log n)$$

3.4 Master's Theorem

Master's Theorem provides a direct method for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where

- $a \geq 1$
- $b > 1$

Case 1

If

$$f(n) = O(n^{\log_b a - \epsilon})$$

for some $\epsilon > 0$

Then

$$T(n) = \Theta(n^{\log_b a})$$

Example

$$T(n) = 4T(n/2) + n$$

$$a = 4, b = 2$$

$$n^{\log_2 4} = n^2$$

Since

$$n = O(n^{2-\epsilon})$$

Case 1 applies.

$$T(n) = \Theta(n^2)$$

Case 2

If

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

Then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Example

$$T(n) = 2T(n/2) + n$$

$$a = 2, b = 2$$

$$n^{\log_2 2} = n$$

$$f(n) = n$$

Case 2 applies.

$$T(n) = \Theta(n \log n)$$

Case 3

If

$f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity condition holds,

Then

$$T(n) = \Theta(f(n))$$

Example

$$T(n) = 2T(n/2) + n^2$$

$$n^{\log_2 2} = n$$

$$f(n) = n^2$$

Case 3 applies.

$$T(n) = \Theta(n^2)$$

1. ANALYSIS OF RECURSIVE ALGORITHMS**1 INTRODUCTION**

Many algorithms solve a problem by dividing it into smaller subproblems, solving each recursively, and combining the results.

Examples:

- Merge Sort
- Quick Sort
- Binary Search

- Strassen Matrix Multiplication
- Tower of Hanoi

The running time of such algorithms is represented using **Recurrence Relations**.

Why Analyze Recursive Algorithms?

To determine:

- Running time
- Efficiency
- Scalability
- Best, Average and Worst Case Complexity

Example: Binary Search

Algorithm:

1. Compare target with middle element.
2. If equal → return.
3. If smaller → search left half.
4. If larger → search right half.

Since each step halves the input:

$$T(n) = T(n/2) + 1$$

2. RECURRENCE EQUATIONS

Definition

A recurrence equation defines a function in terms of its values on smaller inputs.

General Form

$$T(n) = aT(n/b) + f(n)$$

Where:

Symbol	Meaning
a	Number of recursive calls
n/b	Size of each subproblem
f(n)	Cost of division and combination

Example 1: Factorial

$$T(n) = T(n - 1) + 1$$

Example 2: Binary Search

$$T(n) = T(n/2) + 1$$

Example 3: Merge Sort

$$T(n) = 2T(n/2) + n$$

Example 4: Tower of Hanoi

$$T(n) = 2T(n - 1) + 1$$

3. SOLUTION OF RECURRENCE EQUATIONS

Methods:

1. Iteration Method
2. Recursion Tree Method
3. Substitution Method
4. Master's Theorem

4. ITERATION METHOD**Definition**

Repeatedly expand the recurrence until a pattern emerges.

Example 1

Solve: $T(n) = T(n - 1) + 1$

Step 1 : $T(n) = T(n - 1) + 1$

Step 2: $= T(n - 2) + 1 + 1$

Step 3: $= T(n - 3) + 1 + 1 + 1$

Continuing: $= T(n - k) + k$

When $n - k = 1$

$$k = n - 1$$

Therefore $T(n) = T(1) + (n - 1)$

Assume: $T(1) = 1$

Hence: $T(n) = n$

Complexity : $O(n)$

Example 2: Binary Search

$$T(n) = T(n/2) + 1$$

Expand:

$$\begin{aligned} &= T(n/4) + 2 \\ &= T(n/8) + 3 \\ &= T(n/2^k) + k \end{aligned}$$

Stop when:

$$\begin{aligned} n/2^k &= 1 \\ 2^k &= n \\ k &= \log n \end{aligned}$$

Thus:

$$T(n) = O(\log n)$$

5. RECURSION TREE METHOD

Definition

A recurrence is represented as a tree.

Each node shows work performed at that recursive call.

Total cost = Sum of costs at all levels.

Example: Merge Sort

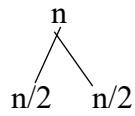
$$T(n) = 2T(n/2) + n$$

Level 0

n

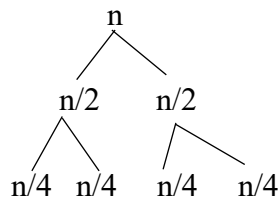
Cost = n

Level 1



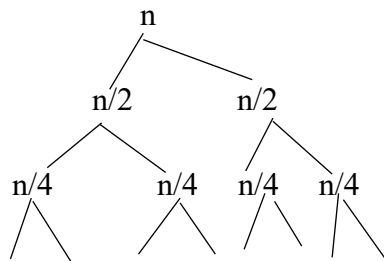
Cost: $n/2 + n/2 = n$

Level 2



Cost: $4 \times n/4 = n$

Level 3



Cost = n

Number of Levels

$$\log n$$

Total Cost

$$n + n + n + \dots + n$$

($\log n$ times) $T(n) = n \log n$

Complexity $O(n \log n)$

6. SUBSTITUTION METHOD

Definition

1. Guess the solution.
2. Use induction to prove correctness.

Example

$$T(n) = 2T(n/2) + n$$

Guess: $T(n) \leq cn \log n$

Assume: $T(n/2) \leq c(n/2) \log (n/2)$

Substitute:

$$\begin{aligned} T(n) &= 2[c(n/2) \log (n/2)] + n \\ &= cn(\log n - 1) + n \\ &= cn \log n - cn + n \end{aligned}$$

For sufficiently large c ,

$$T(n) \leq cn \log n$$

Therefore $T(n) = O(n \log n)$

7. MASTER'S THEOREM

Applicable for:

$$T(n) = aT(n/b) + f(n)$$

Case 1

If

$$f(n) = O(n^{\log_b a - \epsilon})$$

Then

$$T(n) = \Theta(n^{\log_b a})$$

Example

$$T(n) = 4T(n/2) + n$$

$$a = 4$$

$$b = 2$$

$$n^{\log_2 4} = n^2$$

Since $n < O(n^2)$

Case 1 applies. $T(n) = \Theta(n^2)$

Case 2

If $f(n) = \Theta(n^{\log_b a} \log^k n)$

Then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Example $T(n) = 2T(n/2) + n$

$$a = 2$$

$$b = 2$$

$$n^{\log_2 2} = n$$

Case 2 applies. $T(n) = \Theta(n \log n)$

Case 3

If

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

Then

$$T(n) = \Theta(f(n))$$

Example

$$T(n) = 2T(n/2) + n^2$$

$$n^2 > n$$

Case 3 applies.

$$T(n) = \Theta(n^2)$$

MASTER THEOREM SUMMARY TABLE

Recurrence	Complexity
$T(n)=T(n/2)+1$	$O(\log n)$
$T(n)=2T(n/2)+n$	$O(n \log n)$
$T(n)=4T(n/2)+n$	$O(n^2)$
$T(n)=2T(n/2)+n^2$	$O(n^2)$

8. AVL TREES**Introduction**

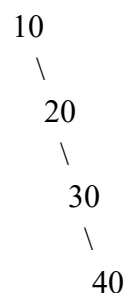
AVL Tree is a self-balancing Binary Search Tree.

Introduced by:

- Adelson-Velsky
- Landis

Need for AVL Trees

Normal BST may become:



Height = n

Search Complexity: $O(n)$

AVL maintains balance.

AVL Property

For every node:

$$| \text{Height}(\text{Left}) - \text{Height}(\text{Right}) | \leq 1$$

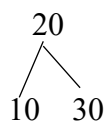
BALANCE FACTOR

$$BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$$

Possible values:

- -1
- 0
- +1

Balanced AVL Tree:



BF = 0

9. AVL ROTATIONS

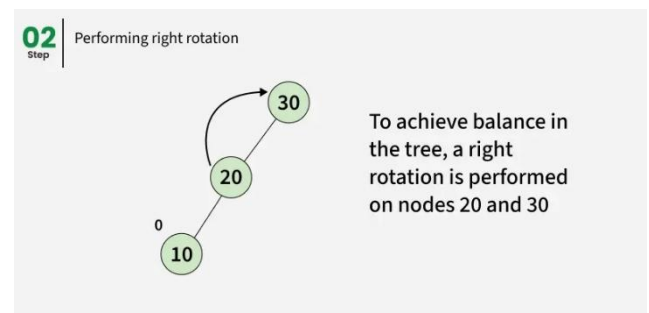
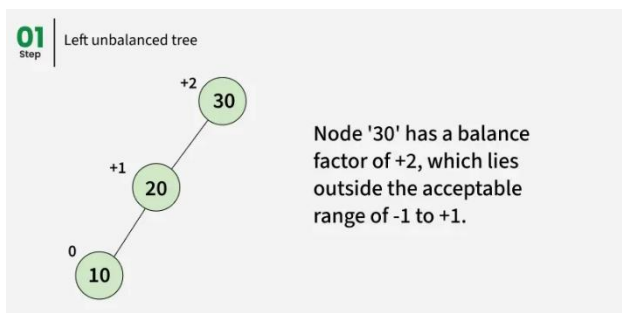
Four rotations:

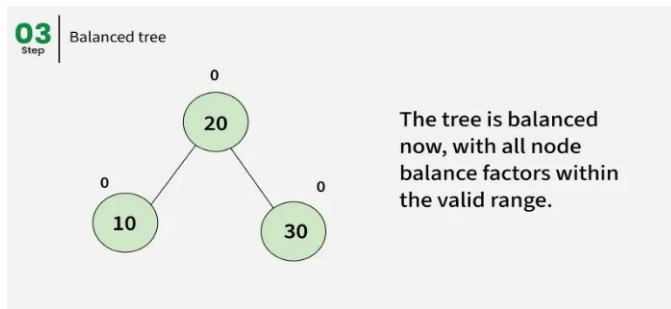
1. LL Rotation
2. RR Rotation
3. LR Rotation
4. RL Rotation

10. LL ROTATION (RIGHT ROTATION)

Cause

Insertion in left subtree of left child.

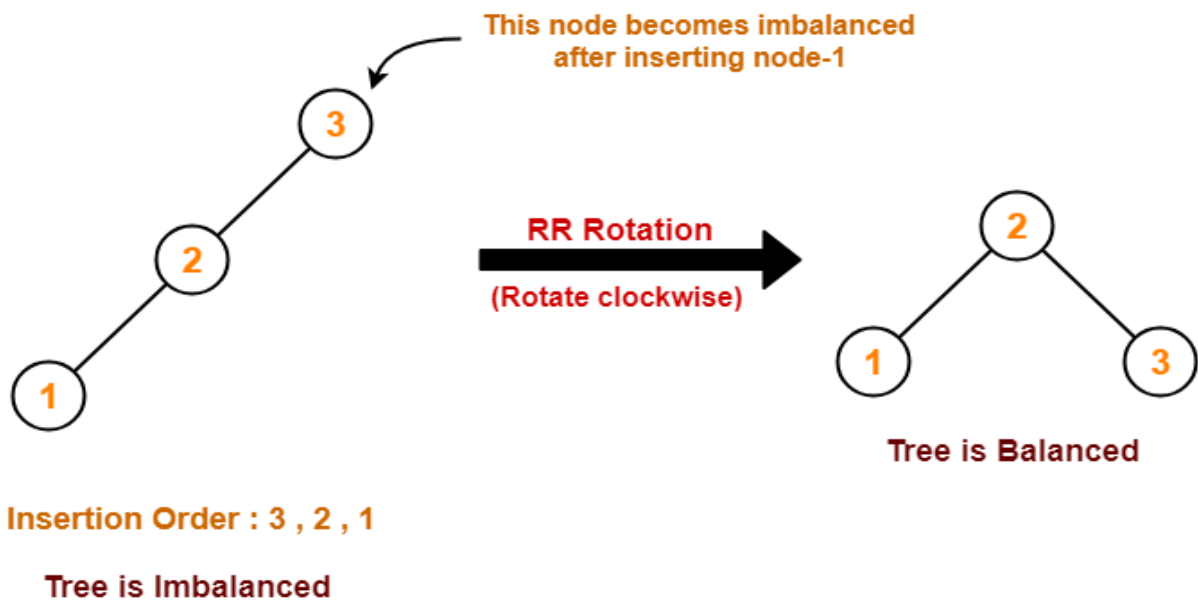




11. RR ROTATION (LEFT ROTATION)

Cause

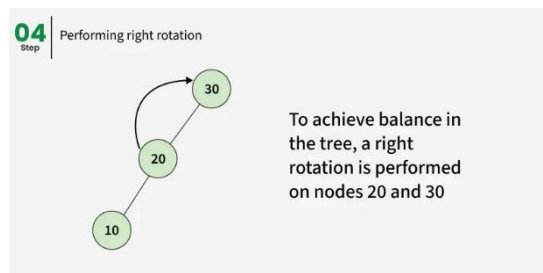
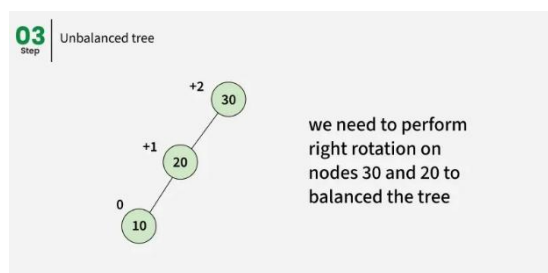
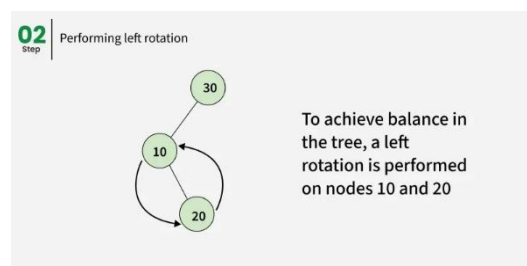
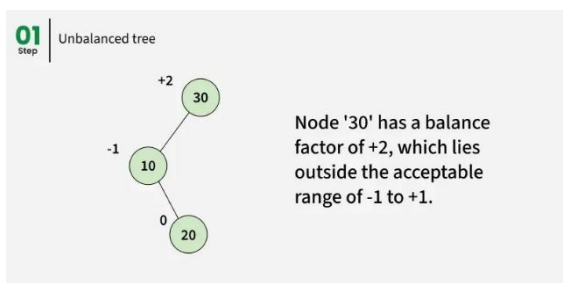
Insertion in right subtree of right child.

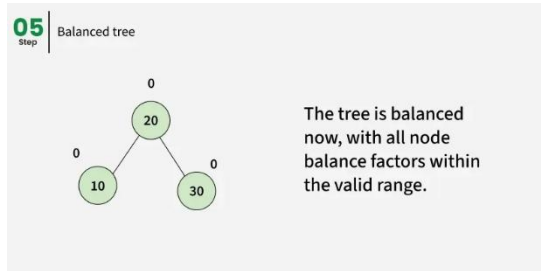


12. LR ROTATION

Cause

Insertion in right subtree of left child.

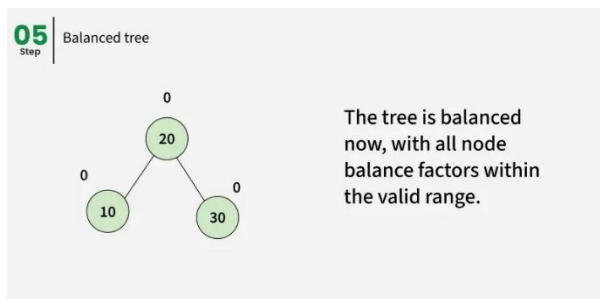
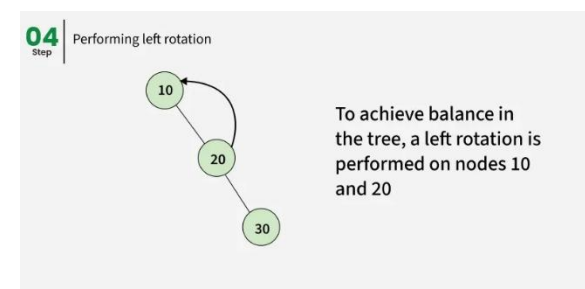
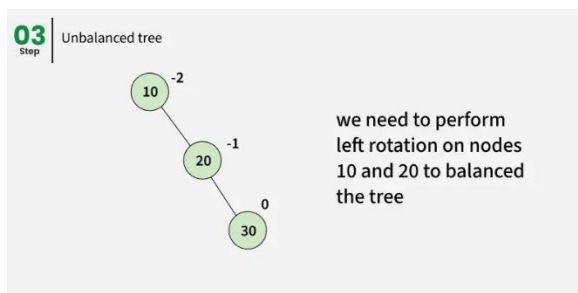
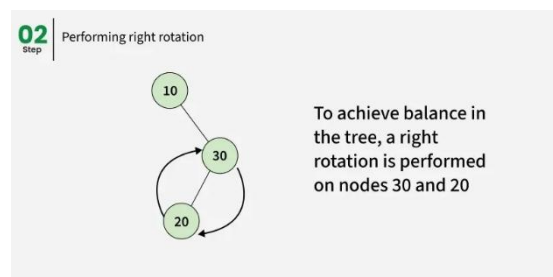
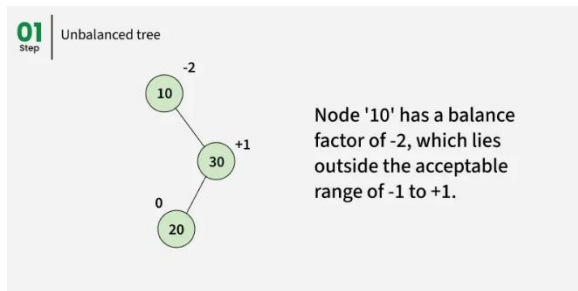




13. RL ROTATION

Cause

Insertion in left subtree of right child.



14. AVL INSERTION

Step 1

Insert node as BST.

Step 2

Update heights.

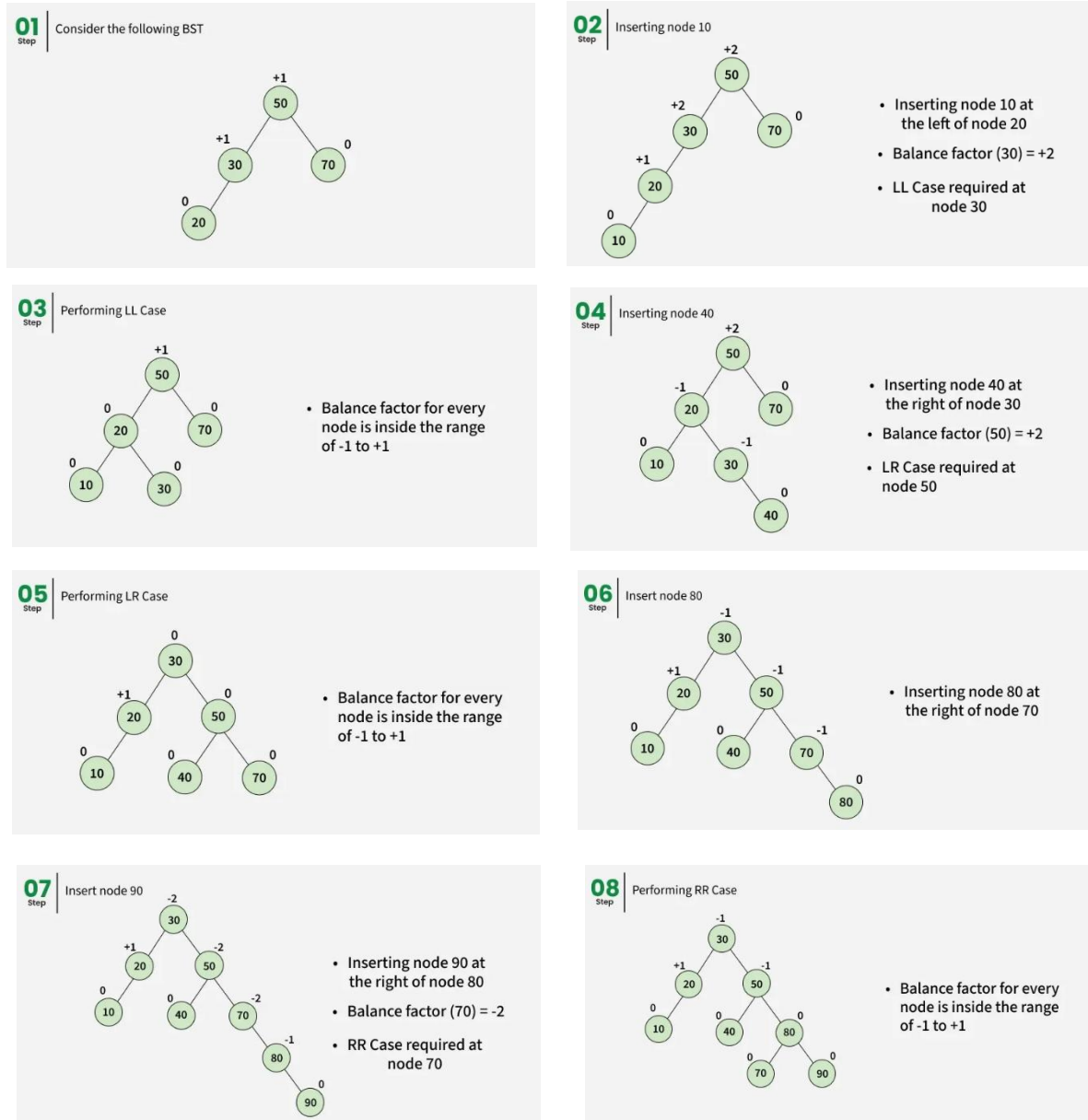
Step 3

Compute Balance Factor.

Step 4

Apply appropriate rotation.

Example



15. AVL DELETION

Deletion follows BST deletion.

Cases:

1. Leaf Node

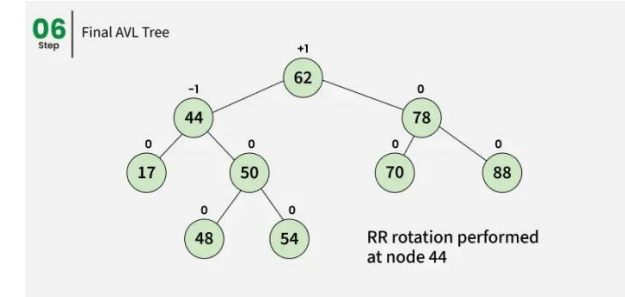
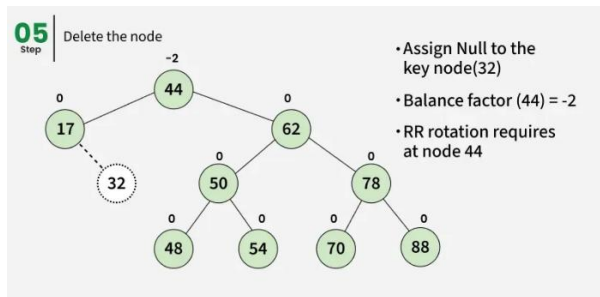
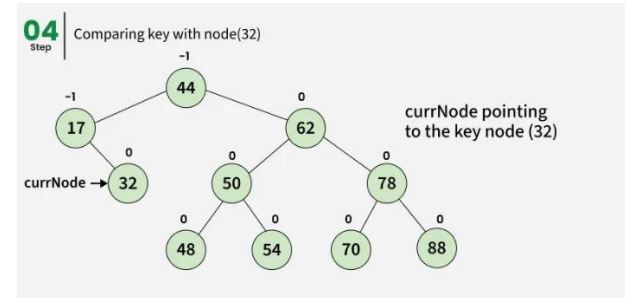
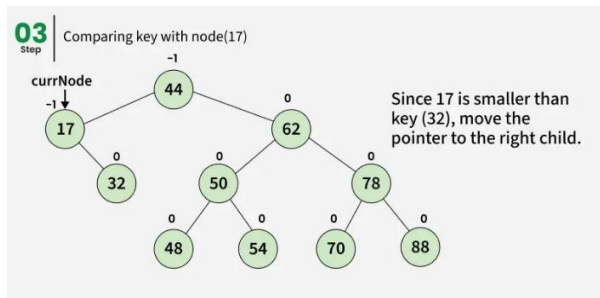
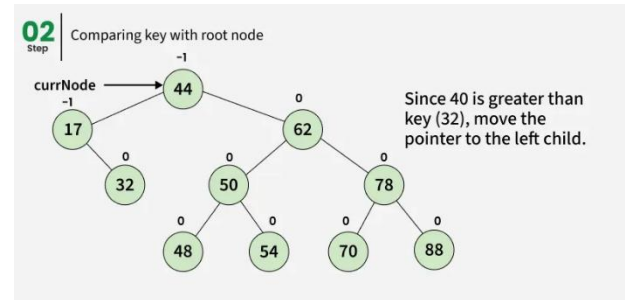
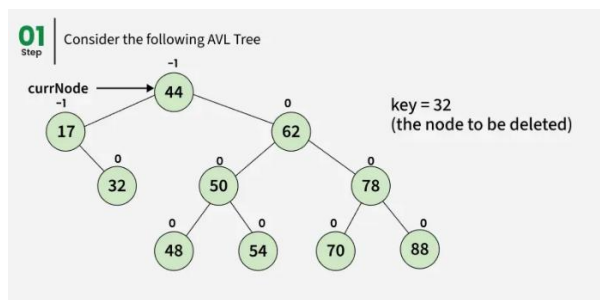
2. One Child

3. Two Children

After deletion:

- Update heights
- Calculate BF
- Rebalance if necessary

Example



AVL COMPLEXITIES

Operation	Time
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$
Rotation	$O(1)$